

Phylogeny: Where Compilers Came From

The deeper history: geographies, characters, and the long development of the idea that a program could translate another program.

The idea

A compiler is a translator. It reads text in one language and produces equivalent behavior in another — machine instructions, or text in a lower-level language, or both. That sounds obvious now. In 1950 it was not obvious. It was, to most working programmers, impossible.

Before compilers, programs were written in machine language: sequences of numbers that corresponded directly to processor instructions. Programming meant knowing your machine's instruction set, its register names, its memory layout. Every machine was its own world. Moving a program from one machine to another meant rewriting it from scratch.

The insight that changed this: the machine does not need to be programmed by a human who understands its every particular. It can be programmed by *another program* that translates from something more human-readable. The machine is a patient entity. You can put a translator in front of it.

This seems obvious in retrospect. It took until 1952 for anyone to build it, and when it was built, people didn't believe it worked.

Geographies

Bletchley Park — Buckinghamshire, England

The war came first. Bletchley Park, 1939 to 1945: the Government Code and Cypher School, where British mathematicians broke the German Enigma and Lorenz ciphers. Alan Turing worked here. He designed the Bombe — an electromechanical device that searched cipher keys at scale — and contributed to the Colossus project, one of the first programmable electronic computers.

But Turing's foundational contribution predated Bletchley by three years. In 1936 he published "On Computable Numbers, with an Application to the Entscheidungsproblem." The paper introduced the

Turing machine: an abstract device that reads symbols from a tape and writes symbols to a tape, according to a finite set of rules. Simple in description. Universal in scope: any computation that can be precisely described can be performed by a Turing machine. This was the proof that computation has a definite shape — that it is not tied to any particular physical implementation, any particular machine, any particular language.

Before you can ask how to translate between programming languages, you have to establish that all programs computing the same function are equivalent. Turing did that in 1936. The compiler idea depends on it.

Turing was killed by the British government in 1954 — sentenced to chemical castration for homosexuality, dead of cyanide poisoning at 41. He was pardoned posthumously in 2013. He never saw a compiler run.

Murray Hill, New Jersey — Bell Labs

Bell Telephone Laboratories at Murray Hill, New Jersey is the geography that matters most for this unit. Bell Labs was the research division of AT&T, and for most of the twentieth century it was the most productive industrial research institution on earth. Transistor (1947). Information theory, Shannon (1948). Unix (1969). C (1972). The laser. The solar cell. Seven Nobel Prizes.

The culture has been described as: the smartest people you have ever met, no deadlines, doors left open. Whether this is nostalgia or an accurate description depends on who you ask, but the output is not disputed.

Ken Thompson arrived in 1966. Dennis Ritchie in 1967. Brian Kernighan in 1969. Douglas McIlroy, who invented the Unix pipe, had been there since 1958. These people worked in adjacent offices for more than a decade. They talked every day.

They were building Unix when they built C. The two were inseparable from the beginning. C was designed to write Unix, and Unix was rewritten in C as soon as C could compile enough of it to make this possible. The language and the operating system defined each other.

Berkeley — University of California

UC Berkeley received a Unix license from AT&T in 1974. Berkeley's computer science department — particularly Bill Joy, Eric Allman, Kirk McKusick, and the Berkeley Software Distribution group — turned the license into a fork. BSD Unix added virtual memory, TCP/IP networking, an improved filesystem, and a quality of engineering that influenced Unix for decades.

Bill Joy wrote vi at Berkeley in 1976. He wrote the C shell. When DARPA needed TCP/IP software for the early internet, they funded Berkeley. The 4.2BSD release in 1983 included the first complete TCP/IP implementation, and the internet ran on it.

Berkeley is also where the legal battles happened. When AT&T asserted its copyright over Unix source in the early 1990s, the resulting litigation was settled in a way that freed most of BSD. FreeBSD, NetBSD, and OpenBSD descend directly. Darwin — the core of macOS — is largely derived from BSD. The phone in your pocket, if it is an iPhone, runs code that descends from Berkeley.

Cambridge, Massachusetts — MIT

MIT's AI Lab, through the 1960s and 1970s, was the other major node of American computing culture. John McCarthy invented Lisp there in 1958. The AI Lab ran on ITS — the Incompatible Timesharing System — a hacker-built operating system where the norm was shared source, open machines, and programs that anyone could read and modify. It was, by default, what Stallman later had to fight to preserve.

Richard Stallman arrived at the AI Lab in 1971 as a programmer, not a student. He spent the 1970s writing Emacs — an editor designed to be infinitely extensible from within itself, modifiable while it was running, open to anyone who wanted to read or change it. When the AI Lab culture changed in the early 1980s and proprietary software arrived, Stallman had a word for what was being lost: freedom. He left and tried to rebuild it from scratch.

The GNU project, the GPL, and GCC all come from Stallman's experience of MIT's AI Lab culture ending. He was trying to institutionalize — in licenses and in law — what had existed informally at MIT for a decade.

Helsinki, Finland

Linus Torvalds was a 21-year-old student at the University of Helsinki in 1991 when he started writing a kernel for his new 386 PC. He had Minix, a small educational Unix clone by Andrew Tanenbaum, but wanted something more capable. He started from scratch.

His August 25, 1991 message to the comp.os.minix newsgroup: *"I'm doing a (free) operating system (just a hobby, won't be big and professional like gnu) for 386(486) AT clones."*

Linux is written entirely in C. It required GCC to compile — Torvalds used GCC-specific extensions that would not work with other compilers. GCC was the only free compiler that could build it. The full circle: Ritchie builds C. Stallman builds the free compiler. Torvalds writes a free kernel in C, using the free compiler. The combination produces an operating system that now runs most of the internet, most

of the world's servers, and all of the world's Android phones.

Characters

Alan Turing (1912–1954)

Bletchley Park; University of Manchester. Founded theoretical computer science before computers existed. Established that all sufficiently powerful computational systems are equivalent — that what matters is the algorithm, not the machine. Cracked the Lorenz cipher with the Colossus team. Proposed the Turing Test. Wrote the first program for a stored-program computer (Manchester, 1948). Killed by the British government for homosexuality. Pardoned posthumously, which is not enough.

No compiler. The theoretical ground everything stands on.

Grace Hopper (1906–1992)

Vassar; Harvard; United States Navy. In 1952, built A-0 — the first compiler, a program that translated mathematical notation into machine code. When she told people she had done this, they refused to believe her. "Nobody believed that I had a running compiler and nobody would touch it," she said. "They told me computers could only do arithmetic." They were wrong. She spent the next decade teaching the computing industry that translation was possible, then necessary, then obvious. She retired as a rear admiral of the United States Navy.

First compiler (A-0, 1952). Proved the idea was real.

John Backus (1924–2007)

IBM, New York City. Led the team that produced FORTRAN in 1957 — the first widely used compiled language for scientific computation. Before FORTRAN, the universal belief among working programmers was that compiled code would always be too slow for real work. FORTRAN proved this wrong. Backus also co-developed Backus-Naur Form (BNF) with Peter Naur — the notation for describing programming language grammars, still used in every language specification written today.

FORTRAN (1957). BNF. Proved compilers could be fast.

John McCarthy (1927–2011)

MIT; Stanford. Invented Lisp in 1958 — the second oldest high-level programming language still in use. Lisp introduced: garbage collection, higher-order functions, the meta-circular evaluator (a Lisp interpreter written in Lisp), and the treatment of code and data as the same kind of thing. McCarthy also coined the term "artificial intelligence." Lisp's descendants — Scheme, Common Lisp, Clojure, Racket — continue to influence language design. The eval/apply cycle in Lisp is a kind of universal compiler; studying it is studying the heart of interpretation itself.

Lisp (1958). The other tradition. Still alive.

Ken Thompson (1943–)

Bell Labs; Google. Co-created Unix with Ritchie. Wrote the first Unix kernel in assembly language on a surplus PDP-7 he found unused at Bell Labs. Built B, the language that preceded C. Wrote the first chess program to achieve master-level play (Belle, 1980). Won the Turing Award alongside Ritchie in 1983. Delivered "Reflections on Trusting Trust" as his Turing Award lecture — eight pages that describe the bootstrapping problem and its implications with a precision that has not been improved upon. Now works at Google on the Go programming language.

Unix. B. "Reflections on Trusting Trust."

Dennis M. Ritchie (1941–2011)

Bell Labs, his entire career. Designed the C language. Wrote the first C compiler. Co-created Unix with Thompson. Co-wrote *The C Programming Language* with Kernighan. Received the Turing Award in 1983. His 1993 paper "The Development of the C Language" is the authoritative account of where C came from and why.

Ritchie was quiet, precise, and rarely wrong. He died in October 2011, two weeks after Steve Jobs, in almost complete public obscurity. The operating system on Jobs's phone was built on Ritchie's work. The invisibility of infrastructure — of what happens below the application — is the systems programmer's condition.

C. The first C compiler. K&R. The ground Unix stands on.

Brian Kernighan (1942–)

Bell Labs; Princeton. Did not design C, but named it, wrote its canonical description alongside Ritchie, and did more than anyone to make it comprehensible to working programmers. Invented AWK (with Weinberger and Aho). Wrote troff, the document formatter that produced the Unix manual pages. The "K" in K&R. His 2019 memoir *UNIX: A History and a Memoir* is the most readable account of the Bell Labs era and the one most likely to give you a clear sense of what it was actually like to be there.

K&R. AWK. The voice of Unix.

Steve Johnson (1941–)

Bell Labs. Wrote PCC in 1977 — the first genuinely portable C compiler and the direct predecessor of the compiler architecture used by everything that came after. Also wrote yacc — Yet Another Compiler Compiler — still used to generate parsers for everything from SQL to programming language interpreters. Johnson's contributions are foundational and underrecognized. The front-end/back-end split is his; we have been living with it ever since.

PCC. yacc. The architecture of all compilers.

Richard M. Stallman (1953–)

MIT AI Lab; Free Software Foundation. Founded the GNU project in 1983, announced by a manifesto posted to net.unix-wizards. Founded the Free Software Foundation in 1985. Wrote the GNU General Public License. Started GCC in 1987. Wrote Emacs. Believes — and has argued consistently for forty years — that software freedom is a moral question, not a preference.

You can disagree with Stallman's politics and still benefit from his work every time you compile a program. GCC is used by billions of devices. The GPL has been the legal instrument that kept major software projects free. His positions are not incidental to his software; they are why the software exists.

GNU. GCC. GPL. Software freedom as an ethical position.

Fabrice Bellard (1972–)

ENST Paris; independent. Wrote TCC. Also wrote QEMU (the hardware emulator that underlies most virtualization software), FFmpeg (the codec library underneath most video software), and JSMPEG (an MPEG decoder in JavaScript). His range and productivity are unusual. His minimalism is consistent: he tends to solve the problem with less, and the solution tends to be readable.

TCC. QEMU. FFmpeg. Minimalism as a methodology.

Linus Torvalds (1969–)

University of Helsinki; Oregon. Linux kernel. Git. Not a compiler writer; a user of everything in this directory. The fact that Linux required GCC-specific extensions — and that GCC was freely available under the GPL — closed the loop that Stallman had been working to close since 1983. GNU needed a kernel; Linux needed a free compiler. They found each other.

Linux. Git. The circle closes.

The long line

```
1936 – Turing: "On Computable Numbers." The theoretical ground.
1942 – Zuse: Plankalkül. First high-level language. Never implemented.
1945 – von Neumann architecture. The stored-program computer.
1952 – Hopper: A-0. First compiler. Nobody believed her.
1957 – Backus: FORTRAN. First widely used compiled language.
1958 – McCarthy: Lisp. The other tradition begins.
1960 – ALGOL 60. Block structure, recursion, scope. Never popular, endlessly influential.
1966 – Thompson arrives at Bell Labs.
1967 – Ritchie arrives at Bell Labs.
1969 – Unix kernel, first version. Written in assembly. B language.
1972 – C. First C compiler. Unix rewritten in C.
1974 – Berkeley receives Unix license. BSD era begins.
1977 – Johnson: PCC. The front/back-end split.
1983 – Stallman: GNU Manifesto. The free software project begins.
1987 – Stallman: GCC 1.0 released.
1991 – Torvalds: Linux kernel. "Just a hobby."
2001 – Bellard: TCC. A compiler written over a weekend.
```

The line from Turing to TCC is sixty-five years. The line from Hopper to TCC is forty-nine years. The line from Ritchie to TCC is twenty-nine years.

You are at the end of that line. You are not behind. You are not catching up. You are arriving, as everyone arrives — at the point where the tradition is handed to you and you decide what to do with it.

Fall is a good time for this. The year is beginning. The compiler was waiting.

Bibliography

Essential — freely available

Ritchie, Dennis M. "The Development of the C Language." *History of Programming Languages II* (HOPL-II, ACM), 1993. The author's own account, short and clear. Widely available online; search the

title.

Thompson, Ken. "Reflections on Trusting Trust." *Communications of the ACM* 27.8 (1984): 761–763. Eight pages. Read them before anything else.

Turing, Alan M. "On Computable Numbers, with an Application to the Entscheidungsproblem." *Proceedings of the London Mathematical Society* 42 (1936): 230–265. Historically essential; mathematically demanding. The introduction is accessible.

Hopper, Grace Murray. "The Education of a Computer." *Proceedings of the ACM*, 1952. The paper in which she describes the first compiler. Available through ACM Digital Library; ask a library for access.

Unix Heritage Society. tuhs.org. V1 through V7 Unix source code, original documentation, and archived manuals. Free and searchable.

Computer History Museum Oral History Collection. computerhistory.org. Video interviews with Thompson, Ritchie, Kernighan, Joy, Stallman, and others. Free.

GNU Compiler Collection Internals Manual. Available at gnu.org/software/gcc/. Free, comprehensive, dense.

Lions, John. *Lions' Commentary on UNIX 6th Edition*. Peer-to-Peer Communications, 1996. Circulated privately for twenty years before legal publication. Widely available as a scan. Essential companion to Unit 01; relevant here for context on the C environment where compilers ran.

Standard references — not free

Kernighan, Brian W. and Dennis M. Ritchie. *The C Programming Language*, 2nd ed. Prentice Hall, 1988. The K&R. If you own one book on C, own this one.

Kernighan, Brian W. *UNIX: A History and a Memoir*. Kindle Direct Publishing, 2019. Readable, inexpensive, authoritative. Kernighan was there.

Aho, Alfred V., Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006. The Dragon Book. Theory. Not required for this unit.

Cooper, Keith D. and Linda Torczon. *Engineering a Compiler*, 2nd ed. Morgan Kaufmann, 2011. More approachable than the Dragon Book. Worth having for the longer view.

Salus, Peter H. *A Quarter Century of UNIX*. Addison-Wesley, 1994. A history of Unix by someone who interviewed the people involved. Out of print but findable.

The growth mindset note

The tradition you are reading spans continents, decades, wars, political movements, legal battles, and several fundamental shifts in what we thought computers were for. Nobody knows all of it. The experts know a deep slice.

Your job is to build your slice. Start with what is in front of you: the two compilers in this directory, the two documents that came before this one. Follow your curiosity into the primary sources. When something surprises you — when a line of code does something unexpected, when a historical detail doesn't fit your model — that surprise is information. It is telling you where your model needs to grow.

This is the work. Not knowing everything. Knowing more tomorrow than you know today.

The year is beginning. You are beginning. That is exactly right.